



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table border="1"><thead><tr><th>Student 1</th><th>Student 2</th></tr></thead><tbody><tr><td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr></tbody></table> Name 1: Chau Ying Jia Metric No: A23CS0213 Name 2: Tan Zhi Ming Metric No: A23CS0189	Student 1	Student 2	<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>
Student 1	Student 2				
<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>				
SUBMISSION DATE:	_____				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax       ; LINE4
    mov var2, bx       ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 (9d) into register BX
LINE3	AX = 0009h BX = 0001h	Exchange the value of register AX and BX
LINE4	AX = 0009h var1 = 0009h	Move the value of register AX into variable var1
LINE5	BX = 0001h var2 = 0001h	Move the value of register BX into variable var2.

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 010F0001 EBX = 00F86000 ECX = 00D010AA EDX = 00D010AA ESI = 00D010AA
EDI = 00D010AA EIP = 00D03666 ESP = 010FFE48 EBP = 010FFE54 EFL = 00000246
```

LINE2:

```
EAX = 00AF0001 EBX = 00900009 ECX = 00D010AA EDX = 00D010AA ESI = 00D010AA  
EDI = 00D010AA EIP = 00D0366D ESP = 00AFF818 EBP = 00AFF824 EFL = 00000246
```

LINE3:

```
EAX = 00AF0009 EBX = 00900001 ECX = 00D010AA EDX = 00D010AA ESI = 00D010AA  
EDI = 00D010AA EIP = 00D0366F ESP = 00AFF818 EBP = 00AFF824 EFL = 00000246
```

LINE4:

```
EAX = 00AF0009 EBX = 00900001 ECX = 00D010AA EDX = 00D010AA ESI = 00D010AA  
EDI = 00D010AA EIP = 00D03675 ESP = 00AFF818 EBP = 00AFF824 EFL = 00000246
```

Name	Value	Type
 ax	0x0009	unsigned ..
 bx	0x0001	unsigned ..
 var1	0x0009	unsigned ..
 var2	0x0009	unsigned ..

LINE5:

```
EAX = 00AF0009 EBX = 00900001 ECX = 00D010AA EDX = 00D010AA ESI = 00D010AA  
EDI = 00D010AA EIP = 00D0367C ESP = 00AFF818 EBP = 00AFF824 EFL = 00000246
```

Name	Value	Type
 ax	0x0009	unsigned ..
 bx	0x0001	unsigned ..
 var1	0x0009	unsigned ..
 var2	0x0001	unsigned ..

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc
```

```
.data
```

```
Rval DWORD ?
```

```
Xval DWORD 26
```

```
Yval DWORD 30
```

```
Zval DWORD 40
```

```
.code
```

```
main proc
```

```
    mov eax,Xval        ; LINE1
```

```
    neg eax             ; LINE2
```

```
    mov ebx,Yval        ; LINE3
```

```
    sub ebx,Zval        ; LINE4
```

```
    add eax,ebx         ; LINE5
```

```
    inc eax             ; LINE6
```

```
    mov Rval,eax        ; LINE7
```

```
    exit
```

```
main endp
```

```
end main
```

Answer Q2

a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFEh	Negate register EAX with two's complement
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30) into register EBX.
LINE4	EBX = FFFFFFF6h Zval = 00000028h	Subtract value of register EBX with Zval (40)
LINE5	EAX = FFFFFFFDh EBX = FFFFFFF6h	Add value of register EBX to register EAX
LINE6	EAX = FFFF FF0Dh	Increment EAX by 1
LINE7	EAX = FFFF FF0Dh Rval = FFFF FF0Dh	Move value store in register EAX into variable Rval.

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers
EAX = 0000001A EBX = 00E69000 ECX = 00151005 EDX = 00151005 ESI = 00151005 EDI = 00151005 EIP = 00151015 ESP = 00DDFB0C EBP = 00DDFB18 EFL = 00000246

LINE2:

Registers
EAX = FFFFFFFE6 EBX = 00E69000 ECX = 00151005 EDX = 00151005 ESI = 00151005 EDI = 00151005 EIP = 00151017 ESP = 00DDFB0C EBP = 00DDFB18 EFL = 00000293
0x00154008 = 0000001E

LINE3:

Registers
EAX = FFFFFFFE6 EBX = 0000001E ECX = 00151005 EDX = 00151005 ESI = 00151005 EDI = 00151005 EIP = 0015101D ESP = 00DDFB0C EBP = 00DDFB18 EFL = 00000293
0x0015400C = 00000028

LINE4:

Registers
EAX = FFFFFFFE6 EBX = FFFFFFFF6 ECX = 00151005 EDX = 00151005 ESI = 00151005 EDI = 00151005 EIP = 00151023 ESP = 00DDFB0C EBP = 00DDFB18 EFL = 00000287

LINE5:

Registers
EAX = FFFFFFFDC EBX = FFFFFFFF6 ECX = 00151005 EDX = 00151005 ESI = 00151005 EDI = 00151005 EIP = 00151025 ESP = 00DDFB0C EBP = 00DDFB18 EFL = 00000283

LINE6:

Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00151005 EDX = 00151005 ESI = 00151005 EDI = 00151005 EIP = 00151026 ESP = 00DDFB0C EBP = 00DDFB18 EFL = 00000287
0x00154000 = 00000000

LINE7:

Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00151005 EDX = 00151005 ESI = 00151005 EDI = 00151005 EIP = 0015102B ESP = 00DDFB0C EBP = 00DDFB18 EFL = 00000287

Autos

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
Rval	0xffffffffd	unsigned long
eax	0xffffffffd	unsigned int

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

a) Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 0000 0032h var2 = 0000 000Ah	Multiply the value of var2 (10d) into register EAX
LINE3	EAX = 0000 0046h var3 = 0000 0014h	Add the value of var3 (20d) into register EAX
LINE4	EAX = 0000 0045h var4 = 0000 0045h	Subtract 1 from the value of register and move EAX into variable var4.

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 00000005 EBX = 00C38000 ECX = 00AB1005 EDX = 00AB1005 ESI = 00AB1005
EDI = 00AB1005 EIP = 00AB1015 ESP = 00BFF7E4 EBP = 00BFF7F0 EFL = 00000246

0x00AB4004 = 0000000A
```

LINE2:

```
EAX = 00000032 EBX = 00C38000 ECX = 00AB1005 EDX = 00000000 ESI = 00AB1005
EDI = 00AB1005 EIP = 00AB101B ESP = 00BFF7E4 EBP = 00BFF7F0 EFL = 00000202

0x00AB4008 = 00000014
```

LINE3:

```
EAX = 00000046 EBX = 00C38000 ECX = 00AB1005 EDX = 00000000 ESI = 00AB1005  
EDI = 00AB1005 EIP = 00AB1021 ESP = 00BFF7E4 EBP = 00BFF7F0 EFL = 00000202
```

LINE4:

```
EAX = 00000045 EBX = 00C38000 ECX = 00AB1005 EDX = 00000000 ESI = 00AB1005  
EDI = 00AB1005 EIP = 00AB1022 ESP = 00BFF7E4 EBP = 00BFF7F0 EFL = 00000202
```

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include Irvine32.inc
.data
var1 WORD 40
var2 WORD 10
var4 WORD ?
.code
main proc
mov ax,var1          ; LINE1
mov bx,5             ; LINE2
mul bx               ; LINE3
mov bx,var2          ; LINE4
sub bx,3             ; LINE5
div bx               ; LINE6
mov var4,ax          ; LINE7
exit
main endp
end main
```

Answer Q4

a) Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	Move the immediate value (5) into register BX
LINE3	AX = 00C8h BX = 0005h	Multiply the value of register Bx to register AX and store the value in register DX:AX
LINE4	BX = 000Ah var2 = 000Ah	Move the value of var2(10) into register Bx
LINE5	BX = 0007h	Subtract the value in register Bx with an immediate value (3)
LINE6	AX = 001Ch BX = 0007h DX = 0004h	Divide the register AX by register Bx, the quotient is then store in register Ax and the remainder store in register Dx
LINE7	AX = 001Ch var4 = 001Ch	Move the value of register AX into variable var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers
EAX = 00EF0028 EBX = 00C87000 ECX = 00301005 EDX = 00301005 ESI = 00301005 EDI = 00301005 EIP = 00301016 ESP = 00EFFB74 EBP = 00EFFB80 EFL = 00000246

LINE2:

Registers

EAX = 00EF0028 EBX = 00C80005 ECX = 00301005 EDX = 00301005 ESI = 00301005 EDI = 00301005 EIP = 0030101A ESP = 00EFFB74 EBP = 00EFFB80 EFL = 00000246

LINE3:

Registers

EAX = 00EF00C8 EBX = 00C80005 ECX = 00301005 EDX = 00300000 ESI = 00301005 EDI = 00301005 EIP = 0030101D ESP = 00EFFB74 EBP = 00EFFB80 EFL = 00000202

LINE4:

Registers

EAX = 00EF00C8 EBX = 00C8000A ECX = 00301005 EDX = 00300000 ESI = 00301005 EDI = 00301005 EIP = 00301024 ESP = 00EFFB74 EBP = 00EFFB80 EFL = 00000202

LINE5:

Registers

EAX = 00EF00C8 EBX = 00C80007 ECX = 00301005 EDX = 00300000 ESI = 00301005 EDI = 00301005 EIP = 00301028 ESP = 00EFFB74 EBP = 00EFFB80 EFL = 00000202

LINE6:

Registers

EAX = 00EF001C EBX = 00C80007 ECX = 00301005 EDX = 00300004 ESI = 00301005 EDI = 00301005 EIP = 0030102B ESP = 00EFFB74 EBP = 00EFFB80 EFL = 00000202

LINE7:

Registers

EAX = 00EF001C EBX = 00C80007 ECX = 00301005 EDX = 00300004 ESI = 00301005 EDI = 00301005 EIP = 00301031 ESP = 00EFFB74 EBP = 00EFFB80 EFL = 00000202

Name	Value	Type
 ax	0x001c	unsigned short
 var4	0x001c	unsigned short

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
 - b. The divisor is specified as the operand of the DIV instruction.
 - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
 - d. The remainder is stored in DX.
 - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
 - f. If the divisor is 0, a division error occurs.
 - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

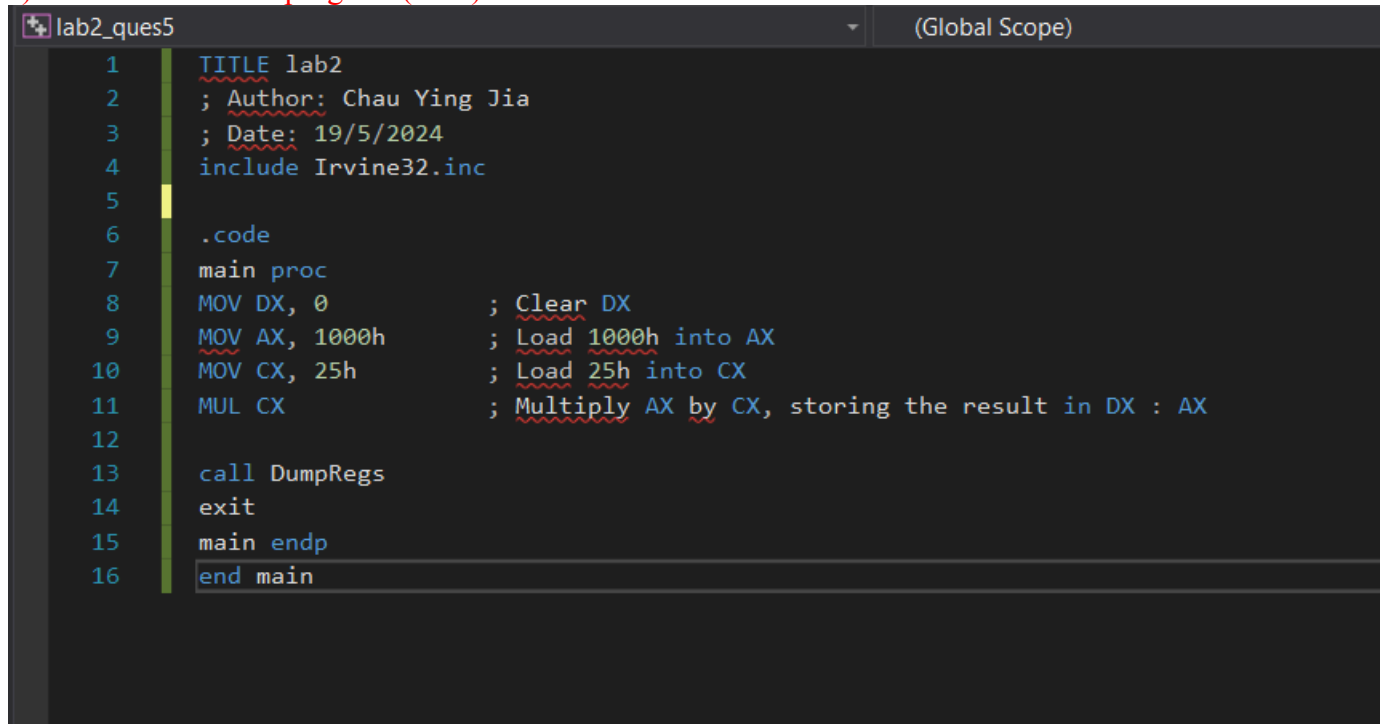
- Write a full program to execute the Code Snippet 1.
- What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX, 0          ; Clear DX
MOV AX, 1000h       ; Load 1000h into AX
MOV CX, 25h         ; Load 25h into CX
MUL CX              ; Multiply AX by CX, storing the result in DX:AX
```

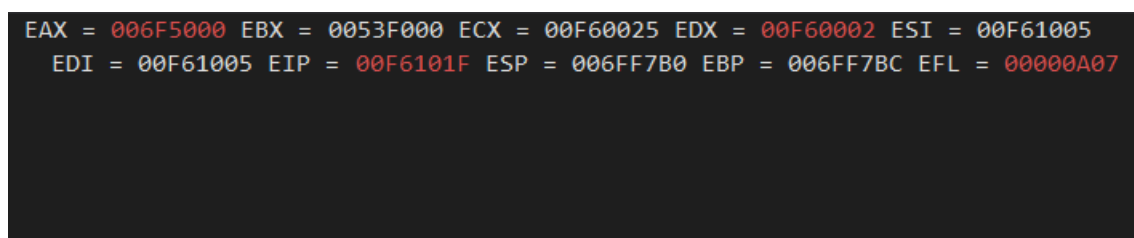
Answer Q5

- Screenshot of full program (.asm) :



```
lab2_ques5 (Global Scope)
1  TITLE lab2
2  ; Author: Chau Ying Jia
3  ; Date: 19/5/2024
4  include Irvine32.inc
5
6  .code
7  main proc
8      MOV DX, 0          ; Clear DX
9      MOV AX, 1000h       ; Load 1000h into AX
10     MOV CX, 25h         ; Load 25h into CX
11     MUL CX              ; Multiply AX by CX, storing the result in DX : AX
12
13     call DumpRegs
14     exit
15     main endp
16     end main
```

- Paste here the screenshot of the final registers' content (DumpReg):



```
EAX = 006F5000 EBX = 0053F000 ECX = 00F60025 EDX = 00F60002 ESI = 00F61005
EDI = 00F61005 EIP = 00F6101F ESP = 006FF7B0 EBP = 006FF7BC EFL = 00000A07
```

Q6. Given the following instructions as is Code Snippet 2.

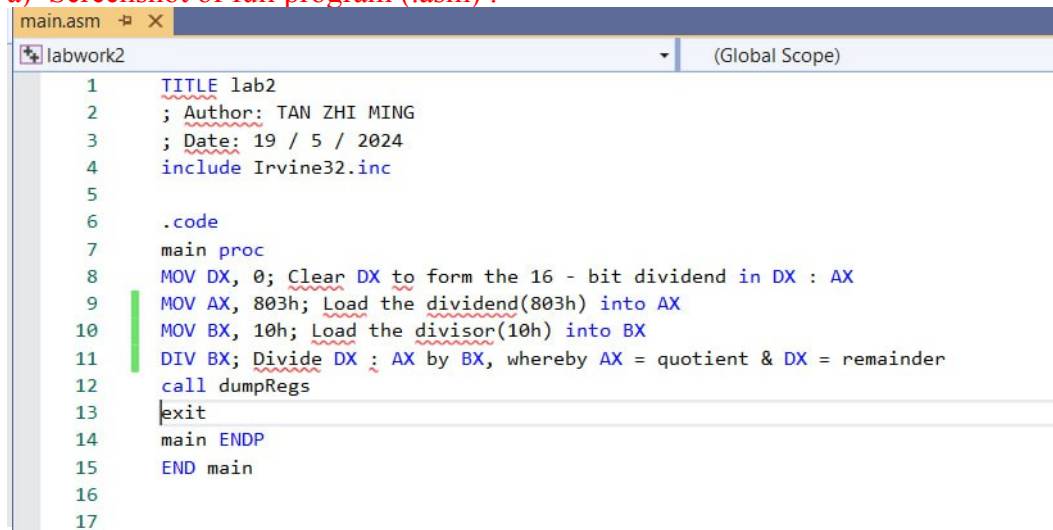
- Write a full program to execute the Code Snippet 2.
- What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BX, 10h    ; Load the divisor (10h) into BL
DIV BX         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

- Screenshot of full program (.asm) :



```
main.asm
labwork2 (Global Scope)
1  TITLE lab2
2  ; Author: TAN ZHI MING
3  ; Date: 19 / 5 / 2024
4  include Irvine32.inc
5
6  .code
7  main proc
8  MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
9  MOV AX, 803h; Load the dividend(803h) into AX
10 MOV BX, 10h; Load the divisor(10h) into BX
11 DIV BX; Divide DX : AX by BX, whereby AX = quotient & DX = remainder
12 call dumpRegs
13 exit
14 main ENDP
15 END main
16
17
```

- Paste here the screenshot of the final registers' content (DumpReg):

